Multi-Vendor E-Commerce Platform & Stripe Payments Engine

Master Technical Handbook: Beginner to Extreme Senior Architecture

Project: Multi-Vendor E-Commerce (Buyer, Vendor, Admin Web)
Engineer: Akshay Chandran P
Tech Stack: Flutter, Dart, Node.js/Express, MongoDB, Stripe PaymentIntent, Webhooks, Cloudinary, AWS SES

Section 1: End-to-End Architecture & Working Explanation

1.1 Beginner Level: How Multi-Vendor Marketplaces Work

In single-seller e-commerce, the store sells its own goods. In a **Multi-Vendor Marketplace** (like Amazon or Etsy): - Multiple independent vendors list and manage products, set stock levels, and dispatch orders. - A buyer can add items from Vendor A and Vendor B into a single cart. - At checkout, a single payment is made by the customer. - The platform splits the payment: taking a platform commission fee (e.g., 10%) and routing the remaining funds to each vendor's account via **Stripe Connect**.

1.2 Extreme Senior Level: Stripe PaymentIntent & 3D Secure 2 Flow



> ***Critical Production Rule:*** *NEVER pass the cart total amount from Flutter to the backend. An attacker can intercept the HTTP request with a proxy (Burp Suite) and change* `$500` *to* `$1`*. The backend MUST fetch product IDs from MongoDB, compute quantities, apply tax/shipping, and create the PaymentIntent with the server-calculated amount.*

1.3 Webhooks vs. Client Callbacks

Client callbacks can fail if the user closes the app, loses internet connection, or runs out of battery right after completing the 3D Secure bank challenge. Therefore, **Order fulfillment MUST NEVER depend on client callbacks**.
The backend listens to Stripe's raw **Webhook** `payment_intent.succeeded`, verifies the cryptographic signature with the webhook signing secret, and fulfills the order asynchronously.

Section 2: Code Audit Findings, Security Risks & Production Fixes

Finding 1: Storing JWT Tokens in Insecure SharedPreferences

- **Problem:** Storing user authentication tokens in Android `SharedPreferences` leaves them exposed in plain text on rooted devices.
- **Production Fix:** Replaced with `flutter_secure_storage` backed by AES-256 hardware keystore encryption, combined with short-lived access tokens (15 mins) and refresh tokens with automatic reuse detection.

Finding 2: Inventory Race Conditions During Flash Sales

- **Problem:** Two users bought the last unit of a product simultaneously. Both checkouts succeeded, creating an oversold order.
- **Production Fix:** Implemented atomic stock deduction using MongoDB conditional operations: `javascript const result = await Product.updateOne( { _id: productId, stock: { $gte: requestedQty } }, { $inc: { stock: -requestedQty } } ); if (result.matchedCount === 0) throw new Error("Out of stock");`

(Handbook continues with full 40 questions covering Stripe Connect, multi-tenant schemas, and Cloudinary pipelines)